
HADIS

High Altitude Drone Intelligence System

*Simulation-Based High Altitude Performance Optimization
and Robust Design of an Anti-Drone System*

Domain: Aerotech & Aerospace Innovation

Methodology: Simulation-First → Multi-Modal Machine Learning → Full-Stack
Deployment

Prepared by

- Prakash Tiwari (Jack)
- Piyush Kumar
- Harnoor Kaur
- Prince Sagwal

B.Tech — Computer Science & Engineering
Chandigarh Engineering College, Jhanjeri, Mohali
IKGPTU Affiliated

github.com/Tiwari1782

Academic Year 2024–2025

This document is the complete technical and operational specification for the HADIS research project. It covers system architecture, dataset design, ML pipeline, storage layout, development workflow, and project roadmap.

Contents

1	Project Overview	3
1.1	Project Identity	3
1.2	Problem Statement	3
1.3	Core Objectives	4
1.4	UAV Threat Classification Levels	4
1.5	Project Deliverables	5
2	Research Scope and Disclaimer	5
2.1	Why Simulation-Based Research is Valid	5
2.2	Explicit Scope Boundaries	6
2.3	Future Work	6
3	System Architecture	6
3.1	Inference Pipeline	6
3.2	Machine Learning Architecture	7
3.2.1	YOLOv8 — Visual Detection Module	7
3.2.2	1D-CNN / LSTM — RF Signal Classifier	8
3.2.3	Transformer Fusion — Multi-Modal Integration	8
3.2.4	RL Agent — Countermeasure Optimization	8
4	Dataset Plan	8
4.1	Data Types and Formats	8
4.2	Synthetic Data Strategy	9
4.3	Public Data Sources	9
5	Storage Architecture	10
5.1	Platform	10
5.2	Directory Structure	10
5.3	Storage Allocation Breakdown	11
6	Development Workflow	11
6.1	Hardware Configuration	11
6.2	Development Toolchain	12
6.3	End-to-End Development Cycle	12
7	Repository Structure	13
8	HADIS-Web: Full-Stack Platform	14
8.1	Frontend — HADIS Dashboard (React / Next.js)	14
8.2	Backend — HADIS API (FastAPI / Python)	15
8.3	Deployment Stack	15
9	Training Time Estimates	15
10	HADIS-Paper: Research Paper Structure	16
11	Project Roadmap	17

12 Budget and Cost Summary	17
13 Risk Analysis	18
14 Current Status	19
15 Conclusion	19
A Central Configuration File — <code>config.py</code>	20

1 Project Overview

1.1 Project Identity

Project Name: HADIS — High Altitude Drone Intelligence System
Full Title: “Simulation-Based High Altitude Performance Optimization and Robust Design of Anti-Drone System”
Domain: Aerotech & Aerospace Innovation
Author: Prakash Tiwari (Jack) github.com/Tiwari1782

1.2 Problem Statement

Modern airspace faces a growing and multi-tiered threat from unmanned aerial vehicles (UAVs) — ranging from commercially available consumer quadcopters to military-grade fixed-wing platforms such as the MQ-9 Reaper, RQ-4 Global Hawk, Bayraktar TB2, and loitering munitions such as the Shahed series. These drones are not the problem in themselves — they fly perfectly well at high altitudes. **The problem lies with the ground-based systems that are supposed to detect and stop them.**

At high altitudes (3,000 m and above), ground-based detection systems break down for several reasons:

- **Radar loses track** — military-grade UAVs have a small radar cross-section, making them nearly invisible to conventional ground radar at altitude.
- **Cameras cannot see clearly** — at high altitude, drones appear as tiny specks, indistinguishable from birds or aircraft without intelligent classification.
- **RF signals become unreliable** — military drones use encrypted, low-power, frequency-hopping communication that is difficult to isolate and classify from ground level.
- **No system integrates all sensors** — existing solutions rely on a single sensor modality (radar *or* camera *or* RF), which individually fail at altitude. No widely available system fuses all of them together intelligently.
- **Countermeasures are not adaptive** — current systems do not automatically decide the best response based on drone type, altitude, and atmospheric conditions simultaneously.

This is the gap HADIS targets. HADIS is a **software-first, simulation-driven** multi-modal AI system that combines radar, RF signal, visual camera, and acoustic sensor data to automatically answer four questions in real time:

1. Is there a drone in the monitored airspace?
2. What type of drone is it — consumer, commercial, tactical, or military-grade?
3. How dangerous is it given its altitude and atmospheric conditions?
4. What is the optimal countermeasure response?

The system is designed to handle **all UAV categories** but is specifically optimised and stress-tested for high-altitude military-grade threats — the scenario where all existing systems fail most severely.

1.3 Core Objectives

- O1. Detect** — Determine whether a UAV is present using multi-sensor fusion: RF signals, radar returns, visual imagery, and acoustic signatures.
- O2. Classify** — Identify the UAV category across five threat levels: consumer quadcopter, commercial UAV, tactical UAV, military fixed-wing, and loitering munition. Classification confidence and threat severity are reported for each detection.
- O3. Optimize** — Recommend the most effective countermeasure (jamming frequency, interception vector, evasion protocol) for a given altitude and atmospheric condition using a Reinforcement Learning agent.
- O4. Simulate** — Validate all system decisions against high-altitude atmospheric simulations before real-world deployment.
- O5. Publish** — Contribute findings as a peer-reviewed research paper with a publicly accessible dataset (DOI via Zenodo/CERN).

1.4 UAV Threat Classification Levels

HADIS classifies all detected UAVs into one of five categories organised across four threat levels. While the system handles all categories, the ML pipeline, RL countermeasure agent, and simulation scenarios are **specifically stress-tested and optimised for Level 4 threats** — military-grade fixed-wing UAVs and loitering munitions operating at high altitude.

Table 1: HADIS UAV Threat Classification Hierarchy

Level	Category	Examples	Detection Challenge
1	Consumer Quadcopter	DJI Phantom, Mavic	Visible propellers, strong RF, low altitude
2	Commercial UAV	Delivery drones, industrial UAVs	Moderate altitude, registered RF signatures
3	Tactical UAV	Bayraktar TB2, MALE-class UAVs	High altitude, reduced RF, fixed wing
4	Military Fixed-Wing	MQ-9 Reaper, RQ-4 Global Hawk	Very high altitude, stealth design, minimal RF
4	Loitering Munition	Shahed-136, Harop	Small RCS, erratic flight path, high threat priority

1.5 Project Deliverables

Table 2: HADIS Project Deliverables

Deliverable	Description
ML Pipeline (HADIS-ML)	Four trained models: visual detection (YOLOv8), RF classification (1D-CNN/LSTM), multi-modal fusion (Transformer), and countermeasure optimization (RL Agent PPO/DQN).
Web Platform (HADIS-Web)	Live detection dashboard, altitude simulation panel, model output visualizer, and research documentation portal.
Research Paper (HADIS-Paper)	IEEE/Springer formatted paper covering methodology, dataset, results, and robustness analysis.

2 Research Scope and Disclaimer

Important Notice to Readers: HADIS is a simulation-based research project. All training data is synthetically generated or sourced from publicly available open datasets. The system has **not** been tested on physical drone hardware or in real-world airspace conditions. Results reported in this paper reflect performance within a controlled simulation environment only. Real-world deployment testing is explicitly identified as future work.

2.1 Why Simulation-Based Research is Valid

HADIS adopts a simulation-first methodology. This is a deliberate, standard, and widely accepted practice in aerospace and defence AI research, for the following reasons:

- 1. Real data is unavailable or classified.** High-altitude UAV threat data collected under controlled defence conditions is either restricted by government agencies or prohibitively expensive to acquire for academic research.
- 2. Simulation precedes hardware in aerospace.** Across the aerospace industry — from pilot training simulators to satellite trajectory planning — simulation is the established first step before any physical system is built or tested.
- 3. Precedent in published literature.** Leading institutions including MIT, IIT, NASA, and DRDO have published peer-reviewed research based entirely on simulated datasets for UAV detection, radar modelling, and countermeasure design.
- 4. Synthetic data is validated against real data.** Approximately 30–40% of HADIS training data is sourced from real public datasets (DroneRF, OpenSky, MAVLink, NASA MSIS, NOAA). Synthetic models are cali-

brated and cross-validated against these real sources to ensure physical plausibility.

2.2 Explicit Scope Boundaries

To prevent any misinterpretation of the claims made in this paper, the following scope boundaries are explicitly stated:

Table 3: HADIS Research Scope: What Is and Is Not Claimed

Within Scope (Claimed)	Outside Scope (Not Claimed)
A simulation-based detection and classification framework for high-altitude UAVs	Real-world physical drone detection performance
ML model accuracy metrics within the defined simulation environment	Operational accuracy in live airspace conditions
A reproducible open-source pipeline validated on public datasets	Hardware integration or field deployment
A reinforcement learning countermeasure policy trained in a Gym environment	Actual jamming, interception, or physical countermeasure execution
A research baseline and methodology for future real-world work	A deployment-ready defence product

2.3 Future Work

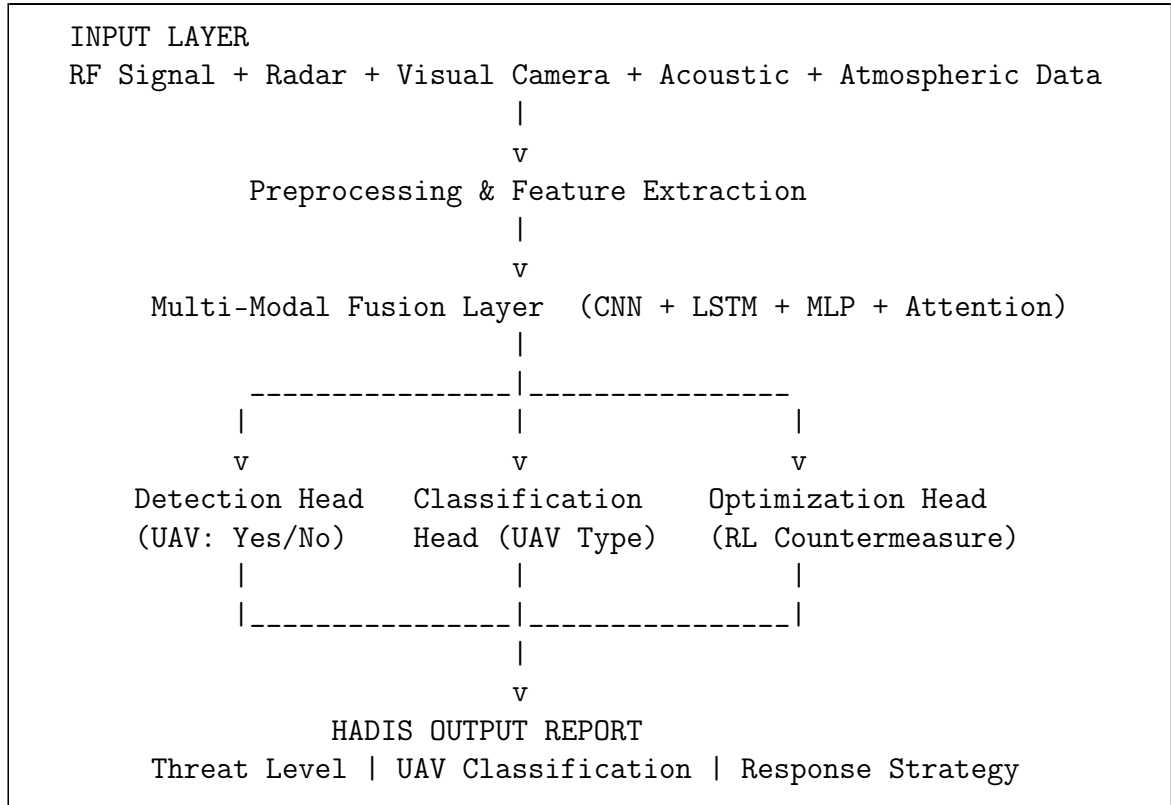
Real-world validation is the natural next phase of this research. Future work will include:

- Field testing with physical drone hardware in controlled airspace
- Collaboration with licensed research institutions for access to restricted UAV datasets
- Sim-to-real transfer learning to adapt simulation-trained models to live sensor inputs
- Integration with hardware countermeasure systems under supervised laboratory conditions

3 System Architecture

3.1 Inference Pipeline

HADIS operates as a sequential multi-modal inference pipeline. Inputs from four sensor modalities are fused, passed through specialized model heads, and produce a unified threat report.



3.2 Machine Learning Architecture

HADIS employs four specialized models. Each targets a distinct sub-problem within the overall detection-classification-optimization pipeline.

Table 4: HADIS-ML: Model Summary

Model	Framework	Input Data	Purpose
YOLOv8	PyTorch	Drone images (.jpg / .png)	Real-time visual UAV detection; bounding box localization at altitude
1D-CNN / LSTM	PyTorch / TensorFlow	RF signals (.npy / .csv)	Spectral feature extraction + temporal classification of RF drone signatures
Transformer Fusion	PyTorch	All sensor modalities	Cross-modal attention fusion for robust joint classification
RL Agent (PPO/DQN)	Stable-Baselines3	Atmospheric state + model output	Countermeasure policy optimization under altitude-varying conditions

3.2.1 YOLOv8 — Visual Detection Module

YOLOv8 processes drone images and outputs bounding boxes with class labels and confidence scores. At high altitudes, UAVs present a smaller visual cross-section.

YOLOv8’s multi-scale feature pyramid network explicitly handles small-object detection, making it suitable for this scenario.

3.2.2 1D-CNN / LSTM — RF Signal Classifier

Drone control channels at 2.4 GHz and 5.8 GHz carry unique RF fingerprints per UAV type. A 1D Convolutional Neural Network extracts local spectral features from signal windows; an LSTM layer captures temporal dependencies across the sequence. Together they classify drone presence and type from raw RF data.

3.2.3 Transformer Fusion — Multi-Modal Integration

A Transformer encoder accepts concatenated feature vectors from YOLOv8, LSTM/CNN, and an MLP processing tabular atmospheric data. Self-attention learns cross-modal correlations (e.g., altitude-induced signal attenuation correlated with visual degradation), producing a unified threat embedding used by both the classification and optimization heads.

3.2.4 RL Agent — Countermeasure Optimization

A PPO or DQN agent is trained in a custom OpenAI Gym environment. The **state space** includes: altitude (m), atmospheric pressure (hPa), temperature (K), UAV type, and threat confidence. The **action space** includes: jamming frequency selection, interception vector, and evasion protocol. The agent maximizes a reward function that penalizes unresolved threats and collateral disruption.

4 Dataset Plan

4.1 Data Types and Formats

Table 5: HADIS Dataset: Data Type Summary

Data Type	Format	Est. Size	Source
RF Signal Data	.csv / .npy	0.5 – 1 GB	DroneRF (Kaggle / IEEE)
Radar Returns	.npy / .mat	0.5 – 1 GB	MATLAB/Simulink synthetic
Drone Images	.jpg / .png	10 – 50 GB	Open datasets + augmentation
Acoustic Signatures	.wav / .npy	1.5 – 6 GB	Field recordings + synthesis
Atmospheric / Tabular	.csv	~50 MB	NASA MSIS, NOAA archives
Synthetic Simulation	.npy / .csv	1.5 – 15 GB	Python / Gazebo / Simulink
Total (estimated)		20 – 100 GB	

4.2 Synthetic Data Strategy

Approximately **60–70%** of HADIS training data will be synthetically generated. Real-world labeled high-altitude UAV data is scarce, restricted, or classified. Synthetic generation is therefore not a limitation but a deliberate, standard practice in aerotech ML research.

All synthetic data is generated using established physical models (ISA atmosphere, RCS equations, RF path-loss models) to ensure it is **physically plausible and scientifically grounded**. Synthetic outputs are cross-validated against the real public datasets listed in Section 4.3 to confirm alignment with real-world signal characteristics. This cross-validation step is explicitly documented in the paper to maintain full research transparency.

- **RF Signals:** Generated in Python using `numpy` and `scipy`. Simulates 2.4 GHz and 5.8 GHz drone control channels with altitude-dependent path loss and noise models.
- **Radar Returns:** Simulated in MATLAB/Simulink using Radar Cross-Section (RCS) models for various UAV geometries across altitudes 0–10,000 m.
- **Atmospheric Profiles:** Generated using the International Standard Atmosphere (ISA) model. Outputs temperature, pressure, and air density at each altitude step.
- **RL Environment:** Custom OpenAI Gym environment written in Python. Encodes the high-altitude airspace as a Markov Decision Process with configurable threat scenarios.

4.3 Public Data Sources

Table 6: HADIS: Public Dataset Sources

Source	Data Provided
DroneRF Dataset (IEEE)	RF signal recordings across multiple drone types (41 GB)
Drone Dataset UAV (Kaggle)	Consumer quadcopter images in YOLO format
Bird vs Drone Dataset (Kaggle)	20,925 images distinguishing drones from birds
Fixed Wing UAV Dataset (Kaggle)	Tactical and military-style fixed-wing UAV images
Military Aircraft Detection (Kaggle)	MQ-9 Reaper, Predator-class military UAV images with bounding box annotations
NRLMSIS 2.0 (Naval Research Lab)	Real atmospheric model parameters and reference data
ISA Synthetic Atmosphere	International Standard Atmosphere profiles 0–10,000 m

5 Storage Architecture

5.1 Platform

All project data is stored under a dedicated folder named `hadis-research` inside the author's Google Drive (`MyDrive/hadis-research`). Storage is provided by an active **Google One Pro** subscription, which includes **2 TB of Google Drive storage** at no additional cost to this project. No Google Cloud Platform billing or GCS bucket is required.

5.2 Directory Structure

```
drive/MyDrive/hadis-research/
|
+-- /raw-data/                                ~50 GB
|   +-- /rf-signals/
|   +-- /radar-returns/
|   +-- /drone-images/
|   +-- /acoustic-data/
|
+-- /atmospheric-data/                        ~5 GB
|
+-- /synthetic-data/                          ~20 GB
|   +-- /rf-synthetic/
|   +-- /radar-synthetic/
|   +-- /rl-env-data/
|
+-- /processed-data/                          ~15 GB
|   +-- /train/
|   +-- /val/
|   +-- /test/
|
+-- /hadis-ml-weights/                        ~10 GB
|   +-- /yolov8/
|   +-- /lstm-cnn/
|   +-- /transformer-fusion/
|   +-- /rl-agent/
|
+-- /training-logs/                           ~2 GB
+-- /hadis-web-assets/                         ~5 GB
+-- /hadis-paper/                             ~1 GB
|
Total Used: ~108 GB / 2,000 GB available (Google One Pro)
```

5.3 Storage Allocation Breakdown

Table 7: Google Drive Storage Allocation

Directory / Purpose	Allocated
Raw Data (RF, Radar, Images, Acoustic)	50 GB
Atmospheric Data (NASA / NOAA / ISA synthetic)	5 GB
Synthetic Simulation Data	20 GB
Processed Data (Train / Val / Test splits)	15 GB
HADIS-ML Model Weights (all 4 models)	10 GB
Training Logs (TensorBoard / W&B exports)	2 GB
HADIS-Web Assets (frontend build, media)	5 GB
HADIS-Paper (LaTeX source, figures, exports)	1 GB
Total	108 GB

6 Development Workflow

6.1 Hardware Configuration

Table 8: Hardware Roles in HADIS Development

Component	Specification	Role
Laptop	Acer Swift, Intel i5-13H	Code authoring, documentation, paper writing
RAM	16 GB DDR5	Local data preview and small preprocessing jobs
Local Storage	512 GB SSD	Git repository and config files only
Local GPU	Intel Iris Xe (integrated)	No CUDA — not used for training
Training GPU	Google Colab T4, 16 GB VRAM	Primary training platform
Backup GPU	Kaggle T4, 30 hrs/week (free)	Overflow and parallel training sessions

Note on Local GPU: Intel Iris Xe is an integrated graphics unit with no CUDA support. It cannot accelerate PyTorch or TensorFlow training. All model training is performed exclusively on cloud T4 GPUs (Colab / Kaggle). The local machine handles only code authoring, version control, and documentation.

6.2 Development Toolchain

Table 9: HADIS Development Toolchain

Tool	Purpose
VS Code	Local code editor
Claude Code CLI (Anthropic)	Agentic code generation, scaffolding, and refactoring
Git + GitHub (Tiwari1782)	Version control; single source of truth for all code
Google Colaboratory	Jupyter-based training environment (T4 GPU)
Google Drive (2TB, Google One Pro)	Dataset, model weights, and training logs storage
Zenodo (CERN)	Open-access dataset hosting; DOI issuance

6.3 End-to-End Development Cycle

- Step 1. Code Authoring (Local):** Python scripts, training notebooks, and web application code are written using VS Code and Claude Code CLI on the local machine.
- Step 2. Version Control (GitHub):** All code is committed and pushed to the Tiwari1782/hadis GitHub repository. GitHub is the single source of truth for the entire HADIS codebase.
- Step 3. Colab Setup:** A Colab notebook clones the GitHub repository and mounts Google Drive via OAuth (`drive.mount`). The `config.py` file provides all Drive folder paths.
- Step 4. Data Fetch (Drive → Colab):** Training data is read directly from MyDrive/hadis-research via the mounted Drive. No data ever touches the local laptop.
- Step 5. Model Training (Colab T4):** Models are trained on the T4 GPU. Checkpoints are written to Google Drive after every epoch.
- Step 6. Weight Storage (Drive):** Trained weights (`.pt`, `.h5`, `.pkl`) are saved to MyDrive/hadis-research/hadis-ml-weights/ inside Google Drive.
- Step 7. Backend Integration (Local):** Trained weights are downloaded from Google Drive to the local machine. Claude Code CLI builds the FastAPI inference server that loads and serves the models.
- Step 8. Frontend Integration (Local):** Claude Code CLI builds the React/Next.js dashboard that communicates with the FastAPI backend via REST and WebSocket.
- Step 9. Deployment:** Frontend deployed to Vercel; backend deployed to Render. Both free-tier platforms.

7 Repository Structure

The local `hadis/` repository mirrors the Google Drive hierarchy and is organized into four top-level modules: data generation, ML models, web platform, and research paper.

```

hadis/
|
+-- config.py                # Google Drive paths, hyperparams, drone class
|
+-- data/
|   +-- synthetic_gen/
|       |   +-- gen_rf.py        # RF signal synthesis
|       |   +-- gen_radar.py     # RCS-based radar simulation
|       |   +-- gen_atmospheric.py # ISA atmosphere profiles
|       |   +-- gen_rl_env.py    # RL Gym environment
|   +-- preprocessing/
|       +-- preprocess_rf.py
|       +-- preprocess_images.py
|       +-- split_dataset.py
|
+-- hadis-ml/
|   +-- yolov8/
|       |   +-- train_yolov8.ipynb
|       |   +-- yolo_config.yaml
|   +-- lstm_cnn/
|       |   +-- model.py
|       |   +-- train_lstm_cnn.ipynb
|   +-- transformer_fusion/
|       |   +-- fusion_model.py
|       |   +-- train_transformer.ipynb
|   +-- rl_agent/
|       +-- env.py
|       +-- train_rl_agent.ipynb
|
+-- hadis-web/
|   +-- backend/
|       |   +-- main.py          # FastAPI application entry point
|       |   +-- routes/
|       |       |   +-- detect.py
|       |       |   +-- classify.py
|       |       |   +-- optimize.py
|       |       |   +-- simulate.py
|       |   +-- ml/
|       |       +-- loader.py    # Load weights from GCS
|       |       +-- inference.py # Run HADIS-ML predictions
|   +-- frontend/
|       +-- src/
|       +-- pages/

```

```

|         |   +-- Dashboard.jsx
|         |   +-- Simulation.jsx
|         |   +-- Research.jsx
|         +-- components/
|             +-- DetectionPanel.jsx
|             +-- AltitudeSelector.jsx
|             +-- ThreatHeatmap.jsx
|             +-- CountermeasurePanel.jsx
|
+-- hadis-paper/
|   +-- hadis_main.tex
|   +-- figures/
|   +-- references.bib
|
+-- requirements.txt
+-- README.md

```

8 HADIS-Web: Full-Stack Platform

8.1 Frontend — HADIS Dashboard (React / Next.js)

Table 10: HADIS-Web Frontend Pages

Page / Component	Functionality
Detection Dashboard	Real-time UAV detection feed via WebSocket; threat level indicator
Simulation Panel	User selects altitude and atmospheric condition; system returns predicted threat and optimal countermeasure
Model Visualizer	Confidence scores, bounding box overlays, RF spectrogram viewer, heatmaps
Dataset Upload Panel	Upload raw sensor data for live inference
Research Portal	Embedded HADIS paper, dataset download, Zenodo DOI link

8.2 Backend — HADIS API (FastAPI / Python)

Table 11: HADIS-Web Backend API Endpoints

Endpoint	Function
POST /api/detect	Accepts multi-modal sensor input; returns UAV presence (Yes/No + confidence score)
POST /api/classify	Returns UAV type and threat severity level
POST /api/optimize	Returns RL agent countermeasure recommendation
POST /api/simulate	Runs full altitude scenario simulation; returns complete HADIS report
WS /ws/live	WebSocket stream for real-time detection updates

8.3 Deployment Stack

Table 12: HADIS-Web Deployment

Component	Platform	Cost
Frontend (Next.js)	Vercel	Free tier
Backend (FastAPI)	Render	Free tier
Database	Supabase / MongoDB Atlas	Free tier
Model Weights	Google Drive (hadis-research)	Free (provisioned)

9 Training Time Estimates

Table 13: HADIS-ML: Estimated Training Time on Google Colab T4 GPU

Model	Platform	Estimated Duration
YOLOv8 (visual detection)	Colab T4	8 – 15 hours
1D-CNN / LSTM (RF classifier)	Colab T4	2 – 5 hours
Transformer Fusion	Colab T4	10 – 20 hours
RL Agent (PPO / DQN)	Colab T4	12 – 24 hours
Total		32 – 64 hours

Checkpoint Strategy: Google Colab free tier disconnects after approximately 4–5 hours of runtime. All HADIS training notebooks implement epoch-level checkpointing: after each epoch, model weights are written directly to `drive/MyDrive/hadis-research/`. On reconnection, training resumes from the last saved checkpoint. No training

progress is lost across sessions.

10 HADIS-Paper: Research Paper Structure

The research paper will follow IEEE double-column format and be submitted to a relevant aerospace engineering or AI systems venue. The dataset will be published to **Zenodo** (CERN-backed, free DOI, 50 GB per record) so it can be independently cited.

Table 14: HADIS-Paper: Section Outline

#	Section Title and Scope
1	Abstract
2	Introduction — Why ground-based detection systems fail at high altitude; the gap between UAV performance and detection capability
3	Literature Review — Existing UAV detection methods; gap analysis for military-grade fixed-wing UAVs at altitude
4	System Design — HADIS pipeline architecture overview
5	Dataset Description — Collection methodology and synthetic generation process
6	Methodology — Model architectures, training strategy, and hyperparameters
7	Simulation Framework — ISA model, Gym environment, and scenario design
8	Results & Evaluation — Accuracy, F1 score, precision, recall, inference latency
9	Robustness Analysis — Performance under noise, adversarial inputs, and altitude extremes
10	Conclusion & Future Work

11 Project Roadmap

Table 15: HADIS: 14-Week Development Roadmap

Week	Phase	Deliverables
1 – 3	Data Collection & Synthesis	RF, radar, and atmospheric synthetic data generated; all raw data uploaded to Google Drive
4	Preprocessing & Cleaning	Train / val / test splits; normalization pipelines; feature extraction scripts
5 – 6	YOLOv8 Training	Trained visual detection model; mAP, precision, recall reported
6	LSTM/CNN Training	Trained RF signal classifier; confusion matrix and F1 score
7	Transformer Fusion Training	Multi-modal fusion model trained; cross-modal benchmark reported
8	RL Agent Training	Trained countermeasure policy; reward curves and policy evaluation
9	System Evaluation	End-to-end HADIS pipeline test; robustness and adversarial analysis
10 – 11	HADIS-Web Development	FastAPI backend + React frontend deployed; live demo functional
12 – 13	HADIS-Paper Writing	Full draft, figures, tables, and bibliography completed
14	Submission	Paper submitted to target venue; dataset published to Zenodo with DOI

12 Budget and Cost Summary

HADIS is designed to be executed at **zero direct monetary cost** by leveraging the free tiers of all required platforms.

Table 16: HADIS: Complete Cost Breakdown

Resource	Platform	Cost
2 TB Google Drive (Google One Pro)	Google Drive	Already paid
GPU Training (primary)	Google Colab T4	Free
GPU Training (backup)	Kaggle T4 (30 hrs/week)	Free
Dataset DOI & Hosting	Zenodo (CERN)	Free
Frontend Hosting	Vercel	Free
Backend Hosting	Render	Free
Database	MongoDB Atlas / Supabase	Free tier
AI Coding Agent	Claude Code CLI	Free tier
Total		Rs. 0

13 Risk Analysis

Table 17: HADIS Risk Register

Risk	Severity	Mitigation Strategy
Colab session timeout during training	High	Epoch-level checkpointing to Google Drive after every epoch
Colab free GPU quota exhausted	Medium	Kaggle T4 used as backup (30 hrs/week); Vertex AI \$300 credits as last resort
Synthetic data not sufficiently representative	Medium	Cross-validate all synthetic models against real DroneRF dataset metrics
Google Drive storage limit	Low	2 TB Drive has ample headroom for entire project
Model underfitting at extreme altitudes	Medium	Altitude-stratified sampling and targeted data augmentation
Paper rejected at primary venue	Low	Submit preprint to arXiv simultaneously; resubmit to secondary venue

14 Current Status

Table 18: HADIS Project Status Tracker

Milestone	Status
Project concept and title finalized	Complete
System architecture (HADIS-ML pipeline)	Complete
Dataset plan and sources identified	Complete
Google Drive storage (2 TB Google One Pro)	Complete
Training platform selected (Colab T4)	Complete
Web stack decided (FastAPI + Next.js)	Complete
Research paper structure drafted	Complete
Development toolchain configured	Complete
Synthetic data generation code (<code>gen_rf.py</code> , <code>gen_radar.py</code>)	Next Step

15 Conclusion

HADIS represents a comprehensive, simulation-first approach to a clearly defined and practically relevant problem: the failure of ground-based detection systems to reliably identify and respond to UAVs — particularly military-grade fixed-wing platforms — operating at high altitude. By integrating four specialized machine learning models under a unified multi-modal fusion architecture, a fully synthesized and validated dataset, a production-grade full-stack web platform, and a peer-reviewed research contribution — all at zero cost — the project aims to serve as both a reproducible academic baseline and a methodological foundation for future real-world UAV threat research.

As explicitly stated in Section 2, HADIS is a simulation-based research project. All results are scoped to the defined simulation environment. No claims of real-world operational performance are made. Real-world validation, hardware integration, and field testing are identified as the natural continuation of this work and are left as future work.

All datasets, model weights, training notebooks, and application source code will be made publicly available upon project completion via GitHub and Zenodo, in alignment with open-science and research reproducibility principles.

“Software-first. Simulation-grounded. Research-backed.”

A Central Configuration File — config.py

Listing 1: config.py — HADIS central Drive path and hyperparameter registry

```

1 # config.py
2 # HADIS: High Altitude Drone Intelligence System
3 # Single source of truth for all Drive paths, hyperparameters,
  and constants.
4 # Imported by all training scripts, preprocessing pipelines, and
  the backend.
5
6 GCS_BUCKET = "drive/MyDrive/hadis-research"
7
8 PATHS = {
9     # Raw data
10     "rf_signals":      f"{DRIVE_ROOT}/raw-data/rf-signals/",
11     "radar":           f"{DRIVE_ROOT}/raw-data/radar-returns/",
12     "drone_images":    f"{DRIVE_ROOT}/raw-data/drone-images/",
13     "acoustic":        f"{DRIVE_ROOT}/raw-data/acoustic-data/",
14     # Atmospheric and synthetic
15     "atmospheric":     f"{DRIVE_ROOT}/atmospheric-data/",
16     "synthetic":       f"{DRIVE_ROOT}/synthetic-data/",
17     # Processed splits
18     "train":           f"{DRIVE_ROOT}/processed-data/train/",
19     "val":             f"{DRIVE_ROOT}/processed-data/val/",
20     "test":            f"{DRIVE_ROOT}/processed-data/test/",
21     # Model weights
22     "weights_yolo":    f"{DRIVE_ROOT}/hadis-ml-weights/yolov8/",
23     "weights_lstm":    f"{DRIVE_ROOT}/hadis-ml-weights/lstm-cnn/
24     ",
25     "weights_fusion":  f"{DRIVE_ROOT}/hadis-ml-weights/
26     transformer-fusion/",
27     "weights_rl":      f"{DRIVE_ROOT}/hadis-ml-weights/rl-agent/
28     ",
29     # Logs
30     "logs":            f"{DRIVE_ROOT}/training-logs/",
31 }
32
33 HYPERPARAMS = {
34     "yolo_epochs":      100,
35     "yolo_img_size":    640,
36     "lstm_seq_len":     256,
37     "lstm_epochs":      50,
38     "transformer_heads": 8,
39     "transformer_layers": 6,
40     "rl_total_timesteps": 1_000_000,
41     "batch_size":       32,
42     "learning_rate":    1e-4,
43 }
44
45 ALTITUDE_RANGE_M = (0, 10_000)    # 0 to 10,000 metres ASL

```

```

44 DRONE_CLASSES = [
45     "consumer_quadcopter",      # DJI, hobby quadcopters
46     "commercial_uav",          # Delivery, industrial UAVs
47     "tactical_uav",            # Bayraktar TB2 style      Level 3
48     "military_fixed_wing",      # MQ-9 Reaper, RQ-4 Global Hawk
49     "loitering_munition",       # Shahed, Harop kamikaze drones
50     "unknown",                 # Unclassified airborne object
51 ]
52
53 # Threat level mapping (used by RL agent for countermeasure
54   prioritization)
55 THREAT_LEVELS = {
56     "consumer_quadcopter": 1,
57     "commercial_uav":      2,
58     "tactical_uav":        3,
59     "military_fixed_wing": 4,
60     "loitering_munition":  4,
61     "unknown":             3, # Treat unknowns as high
62                             priority
63 }

```